

CCS 2105 — Programming Languages

Problem-Solving Exercises on Lua Coroutines

Group Members: Dennis Khenya, Elvis Amani, Kevis Sutter

All 20 questions solved (coroutine.create / resume / yield / status), with written-answer sub-parts included as labelled comments in the code.

```
--[[
=====
DEPARTMENT OF COMPUTER SCIENCE - CCS 2105 (Programming Languages)
1 PROBLEM-SOLVING EXERCISES ON LUA COROUTINES
2 Group Members: Dennis Khenya, Elvis Amani, Kevis Sutter
3
4 All 20 scenarios below are solved using coroutine.create(), coroutine.resume(),
5 coroutine.yield() and coroutine.status() as the core mechanism, per the general
6 instructions. Every scheduler checks coroutine.status() before resuming and
7 checks the boolean returned by coroutine.resume() before trusting the result,
8 so a dead coroutine is never resumed and a runtime error in one task never
9 crashes the whole program.
10
11 Written-answer sub-parts (explain / justify / discuss / compare / critically
12 evaluate) are included as clearly labelled comment blocks directly under the
13 relevant question, exactly where a marker would look for them.
14
15 Run with: lua CCS 2105 Lua Coroutines Solutions.lua
16 (Each question is wrapped in its own run function and called from main() at
17 the bottom, so a marker can also comment out sections to test them one by one.)
18 =====
19 ]]
20
21 math.randomseed(os.time())
22
23 local function divider(title)
24     print("\n" .. string.rep("=", 70))
25     print(title)
26     print(string.rep("=", 70))
27 end
28
29 -----
30 -- QUESTION 1: Intelligent Matatu Terminus Scheduler (20 Marks)
31 -----
32 -- (a) Each route is its own coroutine, closing over its own passenger count
33 --     and waiting-cycle counter - no route can see or corrupt another's state.
34 -- (b) It yields after every single boarding event AND every departure decision.
35 -- (d) Two departure rules are checked every cycle, and once a route's
36 --     coroutine returns it goes "dead" and the scheduler simply skips it
37 --     instead of resuming it again.
38 local function makeRoute(routeName, arrivalRate)
39     return coroutine.create(function()
40         local passengers, waitCycles = 0, 0
41         while true do
42             local boarded = math.random(0, arrivalRate)
43             passengers = passengers + boarded
44             waitCycles = waitCycles + 1
45             coroutine.yield(string.format("[%s] boarding event: +%d pax (total %d), cycle %d",
46                 routeName, boarded, passengers, waitCycles))
47
48             local normalDeparture = passengers >= 8
49             local timeoutDeparture = waitCycles > 3 and passengers >= 5
50             if normalDeparture or timeoutDeparture then
51                 local rule = normalDeparture and "reached 8+ passengers"
52                 or "waited >3 cycles with >=5 passengers"
53                 coroutine.yield(string.format("[%s] DEPARTS with %d passengers (%s)",
54                     routeName, passengers, rule))
55                 return -- coroutine becomes dead here
56             end
57         end
58     end)
59 end
60
61 -- (c) A plain round-robin turn order is used deliberately: every route gets
62 --     exactly one resume per pass over the table, regardless of how close it
63 --     is to departing. This is what stops one busy route (e.g. Ngong) from
64 --     dominating the terminus and starving a quieter one (e.g. Kitengela).
65 local function runQuestion1()
66     divider("QUESTION 1: Matatu Terminus Scheduler")
67     local routes = {
68         {name = "Rongai", co = makeRoute("Rongai", 3)},
69         {name = "Thika", co = makeRoute("Thika", 2)},
70         {name = "Ngong", co = makeRoute("Ngong", 4)},
71         {name = "Kitengela", co = makeRoute("Kitengela", 1)},
72     }
73     local activeRoutes = #routes
74     local cycle = 0
75     while activeRoutes > 0 and cycle < 20 do
76         cycle = cycle + 1
77         for _, route in ipairs(routes) do
78             if coroutine.status(route.co) ~= "dead" then
79                 local ok, message = coroutine.resume(route.co)
80                 if not ok then
81                     print(string.format("ERROR in %s coroutine: %s", route.name, message))
82                 end
83             end
84         end
85     end
86 end
```

```

81         activeRoutes = activeRoutes - 1
82     else
83         print(message)
84         if coroutine.status(route.co) == "dead" then
85             activeRoutes = activeRoutes - 1
86         end
87     end
88 end
89 end
90 end
91 --[[
92 (e) This scheduler is cooperative, not pre-emptive: control only ever
93 leaves a route's coroutine at a coroutine.yield() call that the route
94 itself decided to make. The main loop cannot interrupt a route mid-
95 calculation the way an OS scheduler can interrupt a thread on a timer
96 tick. Because every route yields after exactly one boarding/decision
97 event, turns stay short and fair "by convention", but the guarantee of
98 fairness comes from the routes' own good behaviour, not from forced
99 pre-emption.
100 ]]
101 end
102
103 -----
104 -- QUESTION 2: M-Pesa Transaction Verification and Recovery (20 Marks)
105 -----
106 -- (a) Each transaction is an independent coroutine holding its own stage/state.
107 -- (b) It yields after every processing stage, reporting id + current status.
108 -- (c)/(d) A transaction that fails balance verification returns immediately
109 --         instead of yielding "authorized" - it can never reach authorization,
110 --         and it does so without touching any other transaction's coroutine.
111 local function makeTransaction(txId, amount, balance)
112     return coroutine.create(function()
113         coroutine.yield(txId, "request_received")
114         coroutine.yield(txId, "customer_details_checked")
115
116         if amount > balance then
117             coroutine.yield(txId, "FAILED_insufficient_funds")
118             return
119         end
120         coroutine.yield(txId, "balance_verified")
121         coroutine.yield(txId, "authorized")
122         coroutine.yield(txId, "receipt_generated")
123     end)
124 end
125
126 local function runQuestion2()
127     divider("QUESTION 2: M-Pesa Transaction Verification")
128     local txns = {
129         makeTransaction("TX-1001", 500, 3000),
130         makeTransaction("TX-1002", 8000, 1500), -- deliberately insufficient funds
131         makeTransaction("TX-1003", 250, 250),
132     }
133     local allDead = false
134     while not allDead do
135         allDead = true
136         for _, tx in ipairs(txns) do
137             if coroutine.status(tx) ~= "dead" then
138                 allDead = false
139                 local ok, id, status = coroutine.resume(tx)
140                 if ok then
141                     print(string.format("%s -> %s", id, status))
142                 else
143                     print("Transaction crashed: " .. tostring(id))
144                 end
145             end
146         end
147     end
148     --[[
149     (e) State (amount, balance, which stage it last reached, local variables)
150     is preserved across resumes because a Lua coroutine keeps its OWN call
151     stack. coroutine.yield() doesn't destroy the stack, it just freezes it
152     and hands control back to the caller; coroutine.resume() later restores
153     that exact stack and continues right after the yield() call. This is
154     what lets each transaction "remember" its progress without any manual
155     save/restore code from the programmer.
156     ]]
157 end
158
159 -----
160 -- QUESTION 3: University Course Registration Conflict Resolver (20 Marks)
161 -----
162 -- (a) Registration modelled as a multi-stage coroutine yielding after each check.
163 -- (b) Validation results are sent back into the coroutine via resume(result).
164 local function makeRegistration(studentId, checks)
165     return coroutine.create(function()
166         for _, check in ipairs(checks) do
167             -- yield the stage name we're about to validate, receive the result
168             local passed = coroutine.yield(studentId, "checking_" .. check.name)
169             if not passed then
170                 coroutine.yield(studentId, "REJECTED_at_" .. check.name)
171                 return -- (c) rejected at the exact stage that failed
172             end
173         end
174         coroutine.yield(studentId, "REGISTERED")
175     end)
176 end

```

```

175     end)
176 end
177
178 local function runQuestion3()
179     divider("QUESTION 3: Course Registration Conflict Resolver")
180     -- (d) three students interleaved; each keeps its own checklist and index
181     local students = {
182         {id = "S001", co = makeRegistration("S001", {
183             {name="unit_availability", ok=true}, {name="timetable", ok=true},
184             {name="prerequisites", ok=true}, {name="confirmation", ok=true}}}},
185         {id = "S002", co = makeRegistration("S002", {
186             {name="unit_availability", ok=true}, {name="timetable", ok=false}, -- clash
187             {name="prerequisites", ok=true}, {name="confirmation", ok=true}}}},
188         {id = "S003", co = makeRegistration("S003", {
189             {name="unit_availability", ok=true}, {name="timetable", ok=true},
190             {name="prerequisites", ok=false}, {name="confirmation", ok=true}}}},
191     }
192     -- store each student's own check list + pointer, resume() feeds the
193     -- coroutine the pass/fail result for whatever stage it just announced
194     for _, s in ipairs(students) do s.step = 0 end
195
196     local finished = 0
197     while finished < #students do
198         finished = 0
199         for _, s in ipairs(students) do
200             if coroutine.status(s.co) == "dead" then
201                 finished = finished + 1
202             else
203                 local resultToSend = nil
204                 if s.step > 0 then
205                     -- find the original check list to know if this stage passed
206                     local checksList = ({S001={true,true,true,true}, S002={true,false,true,true}, S003=
207 {true,true,false,true}})[s.id]
208                     resultToSend = checksList[s.step]
209                 end
210                 local ok, id, status = coroutine.resume(s.co, resultToSend)
211                 s.step = s.step + 1
212                 if ok then print(string.format("%s: %s", id, status)) end
213                 if coroutine.status(s.co) == "dead" then finished = finished + 1 end
214             end
215         end
216     end
217     --[[
218     (e) Coroutines suit this workflow because registration is naturally a
219     sequence of dependent stages that must pause to wait on external input
220     (a database check, a clash detector) and then continue exactly where
221     they left off. Coroutines give each student an isolated, resumable
222     "thread of logic" without the complexity, locking or non-determinism
223     of real OS threads, which matters because several students are being
224     processed at once but must never share or corrupt each other's state.
225     ]]
226 end
227
228 -----
229 -- QUESTION 4: Kenya Airways Boarding Priority System (20 Marks)
230 -----
231 -- (a) Each passenger category is its own coroutine.
232 local function makeCategory(name, groupSize)
233     return coroutine.create(function()
234         local boarded = 0
235         while boarded < groupSize do
236             -- (b) resume(false) tells this category "security unavailable"
237             local clearanceOk = coroutine.yield(string.format("%s: waiting to board (%d/%d boarded)", name, boarded,
238 groupSize))
239             if clearanceOk == false then
240                 coroutine.yield(string.format("%s: SUSPENDED - security check unavailable", name))
241             else
242                 boarded = boarded + 1
243                 coroutine.yield(string.format("%s: boarded passenger %d/%d", name, boarded, groupSize))
244             end
245         end
246         coroutine.yield(string.format("%s: category fully boarded", name))
247     end)
248 end
249
250 local function runQuestion4()
251     divider("QUESTION 4: Kenya Airways Boarding Priority")
252     local categories = {
253         {name = "AssistanceNeeded", co = makeCategory("AssistanceNeeded", 3), suspendedRounds = 0},
254         {name = "BusinessClass", co = makeCategory("BusinessClass", 4), suspendedRounds = 0},
255         {name = "FamiliesChildren", co = makeCategory("FamiliesChildren", 3), suspendedRounds = 0},
256         {name = "Economy", co = makeCategory("Economy", 6), suspendedRounds = 0},
257     }
258     local round = 0
259     local anyAlive = true
260     while anyAlive and round < 12 do
261         round = round + 1
262         anyAlive = false
263         for _, cat in ipairs(categories) do
264             if coroutine.status(cat.co) ~= "dead" then
265                 anyAlive = true
266                 -- simulate: BusinessClass randomly loses security clearance in round 2
267                 local clearance = not (cat.name == "BusinessClass" and round == 2)
268                 local ok, msg = coroutine.resume(cat.co, clearance)

```

```

269         if ok then print(msg) else print("Boarding error: " .. tostring(msg)) end
270     end
271 end
272 -- (c)/(d) other categories are resumed in the same round regardless of
273 -- one being suspended - boarding as a whole never stops - and every
274 -- category gets a turn every round, so a suspended one is retried
275 -- next round rather than being abandoned. This IS the fairness fix.
276 end
277 --[[
278 (e) yield() lets a category coroutine pause exactly at the point it
279 discovered it couldn't proceed (no security clearance) while preserving
280 everything it had already done (how many passengers boarded so far).
281 resume() later re-enters it with fresh information (clearance restored),
282 so the category continues from where it paused instead of restarting
283 its whole boarding count from zero.
284 ]]
285 end
286
287 -----
288 -- QUESTION 5: Kericho Tea Factory Production Pipeline (20 Marks)
289 -----
290 local STAGES_Q5 = {"receiving", "weighing", "withering", "drying", "grading", "packaging"}
291
292 -- (a) One coroutine per batch, holding batch id + current stage index.
293 local function makeBatch(batchId)
294     return coroutine.create(function()
295         local stageIndex = 1
296         while stageIndex <= #STAGES_Q5 do
297             local stage = STAGES_Q5[stageIndex]
298             -- (b) yield batch id + current stage after every operation
299             coroutine.yield(batchId, stage)
300
301             if stage == "drying" then
302                 local qualityPassed = math.random() > 0.4 -- simulated inspection
303                 if not qualityPassed then
304                     -- (c) reprocess: step back to withering instead of moving on
305                     coroutine.yield(batchId, "quality_FAILED_reprocessing")
306                     stageIndex = 3 -- back to "withering"
307                     goto continue
308                 end
309             end
310             stageIndex = stageIndex + 1
311             ::continue::
312         end
313         coroutine.yield(batchId, "packaging_complete")
314     end)
315 end
316
317 -- (d) three+ batches scheduled together; state never mixes because each
318 -- batch's stageIndex lives inside its own coroutine's closure, not in a
319 -- shared variable.
320 local function runQuestion5()
321     divider("QUESTION 5: Kericho Tea Factory Pipeline")
322     math.randomseed(7)
323     local batches = {
324         {id = "BATCH-A", co = makeBatch("BATCH-A")},
325         {id = "BATCH-B", co = makeBatch("BATCH-B")},
326         {id = "BATCH-C", co = makeBatch("BATCH-C")},
327     }
328     local finished = 0
329     local safetyCounter = 0
330     while finished < #batches and safetyCounter < 100 do
331         safetyCounter = safetyCounter + 1
332         finished = 0
333         for _, b in ipairs(batches) do
334             if coroutine.status(b.co) == "dead" then
335                 finished = finished + 1
336             else
337                 local ok, id, stage = coroutine.resume(b.co)
338                 if ok then print(string.format("%s -> %s", id, stage)) end
339                 if coroutine.status(b.co) == "dead" then finished = finished + 1 end
340             end
341         end
342     end
343     --[[
344     (e) Without coroutines, "reprocessing" a batch that fails drying would
345     mean manually saving its progress (which stage, which batch id) into an
346     external struct/table and writing a restart function that reconstructs
347     that context. With a coroutine, the batch's stage variable is simply a
348     local variable inside a loop - jumping back to "withering" is one line
349     (resetting stageIndex), and everything else (batch id, loop position)
350     is automatically preserved by the coroutine's own suspended stack.
351     ]]
352 end
353
354 -----
355 -- QUESTION 6: Adaptive Nairobi Traffic Control (20 Marks)
356 -----
357 -- (a) Separate queue state per road, kept inside each road's own coroutine.
358 local function makeRoad(name)
359     return coroutine.create(function()
360         while true do
361             local queueLength = math.random(0, 40)
362             -- (b) yield current traffic state (queue length) every cycle

```

```

363         coroutine.yield(name, queueLength)
364     end
365 end)
366 end
367
368 local function runQuestion6()
369     divider("QUESTION 6: Adaptive Nairobi Traffic Control")
370     math.randomseed(3)
371     local roads = {
372         {name = "ThikaRoad",    co = makeRoad("ThikaRoad"),    cyclesSinceGreen = 0},
373         {name = "MombasaRoad",  co = makeRoad("MombasaRoad"),  cyclesSinceGreen = 0},
374         {name = "NgongRoad",    co = makeRoad("NgongRoad"),    cyclesSinceGreen = 0},
375         {name = "WaiyakiWay",   co = makeRoad("WaiyakiWay"),   cyclesSinceGreen = 0},
376     }
377
378     for cycle = 1, 10 do
379         local queues = {}
380         for _, r in ipairs(roads) do
381             local ok, name, q = coroutine.resume(r.co)
382             queues[r.name] = q
383             r.cyclesSinceGreen = r.cyclesSinceGreen + 1
384         end
385
386         -- (c) score = congestion weight + fairness weight (starvation avoidance)
387         -- (d) the "+ cyclesSinceGreen * 5" term guarantees that a road that
388         -- keeps losing on raw congestion still climbs the score every cycle
389         -- it is skipped, so it can never wait forever.
390         local bestRoad, bestScore = nil, -1
391         for _, r in ipairs(roads) do
392             local score = queues[r.name] + (r.cyclesSinceGreen * 5)
393             if score > bestScore then
394                 bestScore, bestRoad = score, r
395             end
396         end
397         bestRoad.cyclesSinceGreen = 0
398         print(string.format("Cycle %d: %-12s queue=%2d -> GREEN LIGHT (score %d)",
399             cycle, bestRoad.name, queues[bestRoad.name], bestScore))
400     end
401     --[[
402     (e) Pure congestion-based scheduling maximises short-term throughput
403     (always clearing the biggest queue) but can starve a road that is
404     consistently quieter, e.g. Waiyaki Way, even though its drivers still
405     deserve service. Pure round-robin fairness guarantees every road a
406     turn but wastes green-light time on roads with almost no cars while a
407     genuinely congested road backs up. The weighted score above is a
408     deliberate trade-off: congestion dominates in the short run, but the
409     fairness term grows unboundedly the longer a road waits, so it always
410     eventually wins a turn - throughput is favoured most of the time, at
411     the cost of an occasional "wasted" green light on a quiet road.
412     ]]
413 end
414
415 -----
416 -- QUESTION 7: County Hospital Emergency Triage (20 Marks)
417 -----
418 local STAGES_Q7 = {"registration", "triage", "consultation", "discharge"}
419
420 -- (a) Independent coroutine per patient.
421 local function makePatient(patientId, category)
422     return coroutine.create(function()
423         for _, stage in ipairs(STAGES_Q7) do
424             -- (b) yield after each treatment stage, returning control to scheduler
425             coroutine.yield(patientId, category, stage)
426         end
427         coroutine.yield(patientId, category, "discharged")
428     end)
429 end
430
431 local function runQuestion7()
432     divider("QUESTION 7: County Hospital Emergency Triage")
433     local priorityWeight = {critical = 100, urgent = 20, routine = 1}
434     local patients = {
435         {id = "P1", category = "routine", co = makePatient("P1", "routine"), waitTurns = 0},
436         {id = "P2", category = "urgent", co = makePatient("P2", "urgent"), waitTurns = 0},
437         {id = "P3", category = "critical", co = makePatient("P3", "critical"), waitTurns = 0}, -- arrives mid-flow
438         {id = "P4", category = "routine", co = makePatient("P4", "routine"), waitTurns = 0},
439     }
440
441     local finished = 0
442     while finished < #patients do
443         -- (c) score every still-alive patient this round: critical patients
444         -- carry a huge base weight so they are almost always chosen next.
445         -- (d) waitTurns is added on top (ageing) so a routine patient's
446         -- score keeps climbing every round it is passed over, guaranteeing
447         -- it eventually outscores even an urgent case and gets treated -
448         -- this is the starvation-prevention mechanism.
449         local best, bestScore = nil, -1
450         for _, p in ipairs(patients) do
451             if coroutine.status(p.co) ~= "dead" then
452                 local score = priorityWeight[p.category] + p.waitTurns
453                 if score > bestScore then
454                     bestScore, best = score, p
455                 end
456             end
457         end

```

```

457     end
458     if not best then break end
459
460     for _, p in ipairs(patients) do
461         if p ~= best and coroutine.status(p.co) ~= "dead" then
462             p.waitTurns = p.waitTurns + 1 -- everyone not chosen ages by one
463         end
464     end
465
466     local ok, id, cat, stage = coroutine.resume(best.co)
467     if ok then
468         print(string.format("%s (%s) -> %s    [priority score %d]", id, cat, stage, bestScore))
469         best.waitTurns = 0
470     else
471         print("Patient workflow error: " .. tostring(id))
472     end
473
474     finished = 0
475     for _, p in ipairs(patients) do
476         if coroutine.status(p.co) == "dead" then finished = finished + 1 end
477     end
478 end
479
480 --[[
481 (e) A real emergency department has genuinely parallel activity - two
482 nurses can physically triage two different patients at the exact same
483 instant. Lua coroutines only ever run one at a time; "concurrency" here
484 is really fast interleaving on a single logical thread. If two stages
485 truly needed to happen simultaneously (e.g. a lab test running while a
486 doctor consults a different patient), coroutines alone cannot model the
487 true overlap - that would need real OS threads or separate processes.
488 ]]
489
490 -----
491 -- QUESTION 8: Cooperative County Revenue Collection System (20 Marks)
492 -----
493 -- (a) Each service is an independent coroutine holding its own request state.
494 local function makeRevenueService(serviceName, failOnStage)
495     return coroutine.create(function()
496         local stages = {"validation", "payment_verification", "receipt_generation", "completion"}
497         local retries = 0
498         local i = 1
499         while i <= #stages do
500             local stage = stages[i]
501             if stage == failOnStage and retries < 1 then
502                 -- (c) simulated temporary network failure: suspend, don't crash
503                 coroutine.yield(serviceName, "TEMP_FAILURE_at_" .. stage, retries)
504                 retries = retries + 1
505                 -- stay on the same stage index next resume (retry)
506             else
507                 coroutine.yield(serviceName, stage, retries)
508                 i = i + 1
509             end
510         end
511         coroutine.yield(serviceName, "COMPLETED", retries)
512     end)
513 end
514
515 local function runQuestion8()
516     divider("QUESTION 8: County Revenue Collection System")
517     local MAX_RETRIES = 3
518     local services = {
519         {name = "Parking", co = makeRevenueService("Parking", nil), retryCount = 0},
520         {name = "Permits", co = makeRevenueService("Permits", "payment_verification"), retryCount = 0},
521         {name = "LandRates", co = makeRevenueService("LandRates", nil), retryCount = 0},
522     }
523
524     local finished = 0
525     while finished < #services do
526         finished = 0
527         for _, s in ipairs(services) do
528             if coroutine.status(s.co) == "dead" then
529                 finished = finished + 1
530             else
531                 local ok, name, status = coroutine.resume(s.co)
532                 if not ok then
533                     print(string.format("%s crashed: %s", s.name, name))
534                 elseif status:match("^TEMP_FAILURE") then
535                     s.retryCount = s.retryCount + 1
536                     -- (d) requests that fail are retried later, not blocking others,
537                     -- but only up to a sensible cap
538                     if s.retryCount > MAX_RETRIES then
539                         print(string.format("%s: giving up after %d retries", s.name, s.retryCount))
540                     else
541                         print(string.format("%s: %s (retry %d/%d queued)", s.name, status, s.retryCount, MAX_RETRIES))
542                     end
543                 else
544                     print(string.format("%s: %s", s.name, status))
545                 end
546                 if coroutine.status(s.co) == "dead" then finished = finished + 1 end
547             end
548         end
549     end
550     --[[

```

```

551     (e) The retry loop cannot run forever because every failure increments
552     retryCount, and the scheduler stops resuming a service once retryCount
553     exceeds MAX_RETRIES, deliberately abandoning that request rather than
554     resuming it indefinitely. Bounding the retry counter (instead of only
555     checking a boolean "hasFailed" flag) is what turns an open-ended retry
556     into a finite, provably-terminating one.
557 ]]
558 end
559
560 -----
561 -- QUESTION 9: Agricultural Market Decision Support System (20 Marks)
562 -----
563 -- (a) Each market is a coroutine yielding a changing price every round.
564 local function makeMarket(marketName, basePrice, trendBias)
565     return coroutine.create(function()
566         local price = basePrice
567         while true do
568             price = price + math.random(-200, 200) + trendBias
569             coroutine.yield(marketName, price)
570         end
571     end)
572 end
573
574 local function runQuestion9()
575     divider("QUESTION 9: Agricultural Market Decision Support")
576     math.randomseed(11)
577     local markets = {
578         {name = "Kitale", co = makeMarket("Kitale", 4200, 150), history = {}},
579         {name = "Eldoret", co = makeMarket("Eldoret", 4300, 50), history = {}},
580         {name = "Nakuru", co = makeMarket("Nakuru", 4100, 30), history = {}},
581         {name = "Kisumu", co = makeMarket("Kisumu", 4000, -20), history = {}},
582         {name = "Nairobi", co = makeMarket("Nairobi", 4400, 20), history = {}},
583     }
584     local sold = false
585     local round = 0
586     while not sold and round < 15 do
587         round = round + 1
588         local satisfiedThisRound = {}
589         for _, m in ipairs(markets) do
590             if coroutine.status(m.co) ~= "dead" then
591                 local ok, name, price = coroutine.resume(m.co)
592                 -- (b) history kept per market, in that market's own table
593                 table.insert(m.history, price)
594                 print(string.format("Round %d: %-8s price = %d", round, name, price))
595
596                 -- (c) sell condition: price > 4500 AND rose in two consecutive rounds
597                 local h = m.history
598                 if price > 4500 and #h >= 3 and h[#h] > h[#h-1] and h[#h-1] > h[#h-2] then
599                     table.insert(satisfiedThisRound, m)
600                 end
601             end
602         end
603
604         if #satisfiedThisRound > 0 then
605             -- (e) if several markets qualify in the same round, don't just grab
606             -- the first one - pick the genuinely best offer among the winners
607             local chosen = satisfiedThisRound[1]
608             for _, m in ipairs(satisfiedThisRound) do
609                 if m.history[#m.history] > chosen.history[#chosen.history] then
610                     chosen = m
611                 end
612             end
613             print(string.format(">>> SELLING to %s at %d\n", chosen.name, chosen.history[#chosen.history]))
614             sold = true
615             -- (d) stop unnecessary coroutine execution once a market is chosen -
616             -- we simply stop resuming every market's coroutine (they remain
617             -- suspended forever rather than dead, but no more cycles are spent)
618         end
619     end
620     --[[
621     (e) [continued] If several markets satisfy the condition in the very
622     same round, resolving it as "first one checked wins" would make the
623     outcome depend on table iteration order rather than genuine value -
624     that's why the code above collects ALL qualifying markets for that
625     round first, then compares their actual prices before committing to a
626     sale, so the decision is driven by the data, not by scheduling luck.
627     ]]
628 end
629
630 -----
631 -- QUESTION 10: Supermarket Checkout and Stock Consistency (20 Marks)
632 -----
633 -- (b) Stock lives ONLY in the scheduler's table - not duplicated per checkout -
634 -- so there is exactly one source of truth to keep consistent.
635 local sharedStock = {Bread = 5, Milk = 3, Rice = 4}
636
637 -- (a) Each checkout is a coroutine that yields after every scan attempt.
638 local function makeCheckout(counterName, cart)
639     return coroutine.create(function()
640         for _, item in ipairs(cart) do
641             -- ask the scheduler (via yield) whether this item can be sold
642             local granted = coroutine.yield(counterName, "scan_request", item)
643             if granted then
644                 coroutine.yield(counterName, "sold", item)

```

```

645         else
646             coroutine.yield(counterName, "OUT_OF_STOCK", item)
647         end
648     end
649     coroutine.yield(counterName, "checkout_complete", nil)
650 end)
651 end
652
653 local function runQuestion10()
654     divider("QUESTION 10: Supermarket Checkout & Stock Consistency")
655     local counters = {
656         makeCheckout("Till-1", {"Bread", "Milk", "Bread"}),
657         makeCheckout("Till-2", {"Bread", "Rice", "Milk"}),
658         makeCheckout("Till-3", {"Milk", "Bread", "Bread"}),
659     }
660
661     local grantResult = {} -- feeds back into next resume() per counter
662     for i in ipairs(counters) do grantResult[i] = nil end
663
664     local finished = 0
665     while finished < #counters do
666         finished = 0
667         for i, co in ipairs(counters) do
668             if coroutine.status(co) == "dead" then
669                 finished = finished + 1
670             else
671                 local ok, name, action, item = coroutine.resume(co, grantResult[i])
672                 grantResult[i] = nil
673                 if action == "scan request" then
674                     -- (c) decide BEFORE updating stock - this single check-then-
675                     -- deduct step, run one counter at a time, is exactly what
676                     -- prevents stock ever going negative even though several
677                     -- checkouts want the same product "at once".
678                     if sharedStock[item] and sharedStock[item] > 0 then
679                         sharedStock[item] = sharedStock[item] - 1
680                         grantResult[i] = true
681                     else
682                         grantResult[i] = false
683                     end
684                 else
685                     print(string.format("%s: %s %s", name, action, tostring(item)))
686                 end
687                 if coroutine.status(co) == "dead" then finished = finished + 1 end
688             end
689         end
690     end
691     -- (d) proof of consistency: print final stock, none should be negative
692     print("\nFinal stock levels:")
693     for item, qty in pairs(sharedStock) do
694         print(string.format(" %-6s: %d", item, qty))
695         assert(qty >= 0, "stock went negative - consistency violated!")
696     end
697     --[[
698     (e) Because Lua coroutines never truly run at the same instant, the
699     "check stock, then deduct stock" sequence for one scan can never be
700     interrupted halfway by another checkout's scan - there is no window
701     where two tills could both read "1 loaf left" and both decide to sell
702     it. With real OS threads that race condition is very possible and
703     would need a mutex/lock around the check-and-deduct step; cooperative
704     coroutines get that safety for free simply because control only moves
705     between them at explicit, predictable yield points.
706     ]]]
707 end
708
709 -----
710 -- QUESTION 11: Coroutine State Diagnosis and Fault Recovery (20 Marks)
711 -----
712 local function runQuestion11()
713     divider("QUESTION 11: Coroutine State Diagnosis and Fault Recovery")
714
715     local wellBehaved = coroutine.create(function()
716         coroutine.yield("step 1 done")
717         coroutine.yield("step 2 done")
718     end)
719
720     local erroring = coroutine.create(function()
721         coroutine.yield("about to fail")
722         error("simulated fault: divide by nil configuration value")
723     end)
724
725     local stuck = coroutine.create(function()
726         coroutine.yield("waiting for an external event that never arrives")
727         -- deliberately never yields/returns again in this demo
728     end)
729
730     local tasks = {
731         {name = "WellBehaved", co = wellBehaved},
732         {name = "Erroring", co = erroring},
733         {name = "Stuck", co = stuck},
734     }
735
736     for round = 1, 3 do
737         print("-- round " .. round .. " --")
738         for _, t in ipairs(tasks) do

```

```

739 -- (a) coroutine.status() is checked BEFORE every resume attempt
740 local status = coroutine.status(t.co)
741 if status == "dead" then
742     -- (c) never resume a dead coroutine - just report and move on
743     print(string.format("%s: status=dead, skipping", t.name))
744 else
745     -- (b) the boolean success value from resume() is always checked
746     local ok, result = coroutine.resume(t.co)
747     if ok then
748         print(string.format("%s: status=%s, result=%s", t.name, coroutine.status(t.co), tostring(result)))
749     else
750         -- (d) an error inside one coroutine is caught here and
751         -- reported - it does NOT throw an uncaught error that
752         -- would crash this whole scheduling loop
753         print(string.format("%s: ERROR CAUGHT -> %s (now status=%s)", t.name, result,
754 coroutine.status(t.co)))
755     end
756 end
757 end
758 end
759
760 -- attempting to resume the dead WellBehaved coroutine on purpose, safely
761 if coroutine.status(wellBehaved) == "dead" then
762     print("\nDeliberate test: WellBehaved is dead, so we do NOT call resume() on it.")
763 end
764 --[[
765 (e) "suspended" means the coroutine has yielded and is paused mid-
766 execution, still holding its stack, ready to continue from exactly that
767 point on the next resume(). "dead" means the coroutine's function has
768 returned (or errored) and finished permanently - resuming it again is
769 invalid and returns false plus an error message rather than running
770 anything. An "erroring" coroutine is really just a coroutine that
771 became dead via error() instead of a normal return - coroutine.status()
772 reports it as "dead" too, which is exactly why checking the boolean
773 from coroutine.resume() (not just coroutine.status()) is essential to
774 tell "finished successfully" apart from "finished because it crashed".
775 ]]
776 end
777
778 -----
779 -- QUESTION 12: Nairobi Food Delivery Dispatcher (20 Marks)
780 -----
781 -- (a) Each rider is a coroutine that can accept many assignments over its life.
782 local function makeRider(riderName)
783     return coroutine.create(function()
784         local completed = 0
785         while true do
786             -- (b) rider yields while waiting for an assignment/confirmation
787             -- (c) new order data is passed IN through resume(order)
788             local order = coroutine.yield(riderName, "idle_waiting_for_order")
789             if order == "STOP" then
790                 coroutine.yield(riderName, "shift_ended", completed)
791                 return
792             end
793             coroutine.yield(riderName, "assigned", order.id)
794
795             local readyToCollect = coroutine.yield(riderName, "waiting_for_restaurant", order.id)
796             while not readyToCollect do
797                 readyToCollect = coroutine.yield(riderName, "still_waiting_for_restaurant", order.id)
798             end
799
800             completed = completed + 1
801             coroutine.yield(riderName, "delivered", order.id)
802         end
803     end)
804 end
805
806 local function runQuestion12()
807     divider("QUESTION 12: Nairobi Food Delivery Dispatcher")
808     local riders = {
809         {name = "Juma", co = makeRider("Juma"), load = 0},
810         {name = "Achieng", co = makeRider("Achieng"), load = 0},
811         {name = "Kiptoo", co = makeRider("Kiptoo"), load = 0},
812         {name = "Mwangi", co = makeRider("Mwangi"), load = 0},
813     }
814     local orders = {
815         {id = "01", priority = "urgent"},
816         {id = "02", priority = "normal"},
817         {id = "03", priority = "urgent"},
818         {id = "04", priority = "normal"},
819     }
820
821     for _, order in ipairs(orders) do
822         -- (d) urgent orders go to whichever rider currently has the lowest
823         -- load, balancing work while still respecting priority
824         table.sort(riders, function(a, b) return a.load < b.load end)
825         local chosen = riders[1]
826         local ok, name, status, id = coroutine.resume(chosen.co) -- flush idle state
827         print(string.format("%s: %s", name, status))
828         ok, name, status, id = coroutine.resume(chosen.co, order)
829         print(string.format("%s: %s order %s (priority=%s)", name, status, id, order.priority))
830         ok, name, status, id = coroutine.resume(chosen.co, true) -- restaurant confirms ready
831         print(string.format("%s: %s order %s", name, status, id))
832         chosen.load = chosen.load + 1

```

```

833 end
834 for _, r in ipairs(riders) do
835     coroutine.resume(r.co, "STOP")
836 end
837 print("\nFinal rider loads:")
838 for _, r in ipairs(riders) do print(string.format("  %-8s handled %d orders", r.name, r.load)) end
839 --[[
840 (e) yield() sends a value OUT of the coroutine (its current status, e.g.
841 "waiting_for_restaurant") back to the dispatcher, while the argument
842 given to the NEXT resume() call sends a value IN to the coroutine
843 (the new order, or the restaurant's "ready" confirmation), becoming the
844 return value of that same yield() call inside the rider's code. This
845 two-way channel is what lets the dispatcher both observe a rider's
846 state and hand it new work without ever restarting the rider's
847 coroutine from scratch.
848 ]]
849 end
850
851 -----
852 -- QUESTION 13: Drought Monitoring and Water Allocation (20 Marks)
853 -----
854 -- (a) Four monitoring coroutines, one per county.
855 local function makeStation(county, reservoirLevel, consumptionEstimate)
856     return coroutine.create(function()
857         -- (b) yield the county's current readings for the scheduler to collect
858         coroutine.yield(county, reservoirLevel, consumptionEstimate)
859     end)
860 end
861
862 local function runQuestion13()
863     divider("QUESTION 13: Drought Monitoring and Water Allocation")
864     local TOTAL_SUPPLY = 100 -- emergency supply units
865     local stations = {
866         makeStation("Turkana", 12, 30), -- critical: below 20%
867         makeStation("Kitui", 18, 20), -- critical: below 20%
868         makeStation("Garissa", 35, 15), -- not critical
869         makeStation("Machakos", 8, 25), -- critical: below 20%
870     }
871
872     local readings = {}
873     for _, co in ipairs(stations) do
874         local ok, county, level, consumption = coroutine.resume(co)
875         table.insert(readings, {county = county, level = level, consumption = consumption})
876     end
877
878     local MAX_SHARE = TOTAL_SUPPLY * 0.40
879     local allocation = {}
880     local remaining = TOTAL_SUPPLY
881
882     -- (c) priority pass: counties below 20% get served first, capped at 40%
883     table.sort(readings, function(a, b) return a.level < b.level end)
884     for _, r in ipairs(readings) do
885         if r.level < 20 then
886             local requested = math.min(r.consumption, MAX_SHARE)
887             local granted = math.min(requested, remaining)
888             allocation[r.county] = granted
889             remaining = remaining - granted
890         end
891     end
892     -- second pass: whatever is left goes to non-critical counties, same cap
893     for _, r in ipairs(readings) do
894         if r.level >= 20 and remaining > 0 then
895             local requested = math.min(r.consumption, MAX_SHARE)
896             local granted = math.min(requested, remaining)
897             allocation[r.county] = granted
898             remaining = remaining - granted
899         end
900     end
901
902     for _, r in ipairs(readings) do
903         print(string.format("%-9s level=%d%% consumption=%d -> allocated %d units",
904             r.county, r.level, r.consumption, allocation[r.county] or 0))
905     end
906     -- (d) if supply runs out, some critical counties may receive less than
907     -- requested (never more than remaining) - explicitly flagged here:
908     for _, r in ipairs(readings) do
909         if r.level < 20 and (allocation[r.county] or 0) < r.consumption then
910             print(string.format(" WARNING: %s received only %d/%d units requested - supply shortfall",
911                 r.county, allocation[r.county] or 0, r.consumption))
912         end
913     end
914     print(string.format("Remaining unallocated supply: %d units", remaining))
915     --[[
916 (e) Critical counties (reservoir below 20%) are served first because
917 they face the most immediate risk, but the 40% cap prevents any single
918 county - even a desperate one - from draining the entire emergency pool
919 and leaving nothing for the others. Sorting by reservoir level (driest
920 first) before allocating means that if the supply does run short, the
921 counties in the worst position are the ones most likely to receive at
922 least a partial share before the pool empties, which is the fairest
923 ordering available when full coverage for everyone isn't possible.
924 ]]
925 end
926

```

```

927 -----
928 -- QUESTION 14: Kenya Railways Disruption Management (20 Marks)
929 -----
930 -- (a) Each train's coroutine remembers its current station index itself.
931 local function makeTrain(trainName, route)
932     return coroutine.create(function()
933         local pos = 1
934         while pos <= #route do
935             local station = route[pos]
936             -- (b) yield the station we WANT to enter; wait for "go ahead"
937             local canProceed = coroutine.yield(trainName, "arriving_at", station)
938             while not canProceed do
939                 -- (c)/(d) stay suspended right here - journey position (pos)
940                 -- is untouched - until the scheduler resumes with true
941                 canProceed = coroutine.yield(trainName, "held_outside", station)
942             end
943             coroutine.yield(trainName, "departed", station)
944             pos = pos + 1
945         end
946         coroutine.yield(trainName, "journey_complete", nil)
947     end)
948 end
949
950 local function runQuestion14()
951     divider("QUESTION 14: Kenya Railways Disruption Management")
952     local stationDisrupted = {Voi = true} -- Voi is temporarily unavailable
953     local trains = {
954         makeTrain("SGR-101", {"Mombasa", "Voi", "Mtito Andei", "Nairobi"}),
955         makeTrain("SGR-202", {"Nairobi", "Athi River", "Emali", "Voi"}),
956         makeTrain("SGR-303", {"Kisumu", "Nakuru", "Naivasha", "Nairobi"}),
957     }
958
959     local pendingInput = {}
960     local finished = 0
961     local round = 0
962     while finished < #trains and round < 15 do
963         round = round + 1
964         if round == 5 then stationDisrupted.Voi = false end -- station reopens
965
966         finished = 0
967         for i, co in ipairs(trains) do
968             if coroutine.status(co) == "dead" then
969                 finished = finished + 1
970             else
971                 local ok, name, status, station = coroutine.resume(co, pendingInput[i])
972                 pendingInput[i] = nil
973                 if status == "arriving_at" or status == "held_outside" then
974                     if stationDisrupted[station] then
975                         pendingInput[i] = false
976                         print(string.format("%s: %s %s (BLOCKED - station unavailable)", name, status, station))
977                     else
978                         pendingInput[i] = true
979                         print(string.format("%s: %s %s", name, status, station))
980                     end
981                 else
982                     print(string.format("%s: %s", name, status))
983                 end
984                 if coroutine.status(co) == "dead" then finished = finished + 1 end
985             end
986         end
987     end
988     --[[
989     (e) Coroutine continuation means SGR-101 can be held indefinitely at
990     the edge of Voi without any special "save my journey" logic - its
991     position in the route array, and everything else about its trip, is
992     just sitting frozen on its own call stack. Meanwhile SGR-202 and
993     SGR-303 keep running fully independently. The moment Voi reopens, the
994     scheduler simply resumes SGR-101 with `true` and it carries on into
995     the station exactly as if the delay had never happened.
996     ]]
997 end
998
999 -----
1000 -- QUESTION 15: Cooperative University Computing Scheduler (20 Marks)
1001 -----
1002 -- (a) One coroutine per job, yielding after each unit of work.
1003 local function makeJob(jobName, totalSteps)
1004     return coroutine.create(function()
1005         for step = 1, totalSteps do
1006             coroutine.yield(jobName, step, totalSteps)
1007         end
1008         coroutine.yield(jobName, "FINISHED", totalSteps)
1009     end)
1010 end
1011
1012 local function runQuestion15()
1013     divider("QUESTION 15: Cooperative University Computing Scheduler")
1014     local jobs = {
1015         {name = "AIModelSim", co = makeJob("AIModelSim", 6), priority = 3},
1016         {name = "ResultsProcessing", co = makeJob("ResultsProcessing", 4), priority = 1},
1017         {name = "TimetableGen", co = makeJob("TimetableGen", 5), priority = 1},
1018         {name = "LibraryIndexing", co = makeJob("LibraryIndexing", 3), priority = 1},
1019     }
1020

```

```

1021 -- (b) plain round robin pass first, for comparison in part (e)
1022 print("-- Plain round robin --")
1023 local rrJobs = {
1024     makeJob("A", 3), makeJob("B", 3), makeJob("C", 3),
1025 }
1026 local alive = #rrJobs
1027 while alive > 0 do
1028     alive = 0
1029     for _, co in ipairs(rrJobs) do
1030         if coroutine.status(co) ~= "dead" then
1031             local ok, name, step = coroutine.resume(co)
1032             print(string.format("  %s step %s", name, tostring(step)))
1033             if coroutine.status(co) ~= "dead" then alive = alive + 1 end
1034         end
1035     end
1036 end
1037
1038 -- (c) priority-weighted scheduling: high-priority jobs get extra turns
1039 -- per pass, but every job still gets AT LEAST one turn per pass, so
1040 -- nobody is starved outright.
1041 print("\n-- Priority-weighted round robin --")
1042 local finished = 0
1043 while finished < #jobs do
1044     finished = 0
1045     for _, job in ipairs(jobs) do
1046         if coroutine.status(job.co) == "dead" then
1047             finished = finished + 1
1048         else
1049             for turn = 1, job.priority do
1050                 if coroutine.status(job.co) ~= "dead" then
1051                     local ok, name, step, total = coroutine.resume(job.co)
1052                     print(string.format("  %s step %s/%s", name, tostring(step), tostring(total)))
1053                 end
1054             end
1055             -- (d) once a job's coroutine goes dead, it's simply never
1056             -- given another turn table entry - naturally "removed"
1057             if coroutine.status(job.co) == "dead" then finished = finished + 1 end
1058         end
1059     end
1060 end
1061 --[[
1062 (e) Ordinary round-robin gives every job exactly one step per pass,
1063 which is maximally fair but treats an urgent AI simulation the same as
1064 routine library indexing, so the important job finishes no faster than
1065 anything else. The priority-weighted version above gives AIModelSim
1066 three steps for every one step the others get, so it completes in
1067 roughly a third of the passes - at the cost of the low-priority jobs
1068 finishing a little later than they would under pure round-robin. The
1069 trade is intentional: it sacrifices strict equal-turns fairness for
1070 faster completion of the job the university actually cares about most.
1071 ]]
1072 end
1073
1074 -----
1075 -- QUESTION 16: Producer-Consumer Food Distribution System (20 Marks)
1076 -----
1077 -- (a) Producer coroutine generates consignments incrementally via yield().
1078 local function makeProducer()
1079     return coroutine.create(function()
1080         local sizes = {40, 25, 60, 15, 30}
1081         for _, size in ipairs(sizes) do
1082             coroutine.yield(size)
1083         end
1084     end)
1085 end
1086
1087 local function runQuestion16()
1088     divider("QUESTION 16: Producer-Consumer Food Distribution")
1089     -- (b) each market keeps its own remaining capacity
1090     local markets = {
1091         {name = "MarketA", capacity = 50},
1092         {name = "MarketB", capacity = 35},
1093         {name = "MarketC", capacity = 45},
1094     }
1095     local producer = makeProducer()
1096
1097     while coroutine.status(producer) ~= "dead" do
1098         local ok, consignment = coroutine.resume(producer)
1099         if not ok or consignment == nil then break end
1100         print(string.format("Producer released a %d-unit consignment", consignment))
1101
1102         -- (c) choose dynamically: the market with the MOST remaining room
1103         -- that can still take the whole consignment, so we don't fragment
1104         -- a market's space unnecessarily
1105         table.sort(markets, function(a, b) return a.capacity > b.capacity end)
1106         local placed = false
1107         for _, m in ipairs(markets) do
1108             if m.capacity >= consignment then
1109                 m.capacity = m.capacity - consignment
1110                 print(string.format("  -> fully placed in %s (remaining capacity now %d)", m.name, m.capacity))
1111                 placed = true
1112                 break
1113             end
1114         end
1115     end
1116 end

```

```

1115     if not placed then
1116         -- (d) doesn't fit anywhere whole: split across markets by
1117         -- whatever room each one has, largest-room-first, until either
1118         -- the consignment is used up or all markets are full
1119         local remaining = consignment
1120         for _, m in ipairs(markets) do
1121             if remaining <= 0 then break end
1122             local portion = math.min(remaining, m.capacity)
1123             if portion > 0 then
1124                 m.capacity = m.capacity - portion
1125                 remaining = remaining - portion
1126                 print(string.format(" -> split: %d units into %s (remaining capacity now %d)", portion, m.name,
1127 m.capacity))
1128             end
1129         end
1130         if remaining > 0 then
1131             print(string.format(" -> %d units could not be placed anywhere - spoiled/rejected", remaining))
1132         end
1133     end
1134 end
1135 end
1136 --[[
1137 (e) Filling the market with the most spare room first (rather than the
1138 first market in the list) spreads consignments more evenly and avoids
1139 a situation where one market fills up early while another sits mostly
1140 empty. It is efficient because it minimises wasted partial-fills (a
1141 whole consignment is placed in one market whenever physically
1142 possible), and it is fair to the markets because none of them is
1143 consistently starved of stock just because of its position in a table.
1144 ]]
1145 end
1146
1147 -----
1148 -- QUESTION 17: Fraud-Aware Mobile Banking Workflow (20 Marks)
1149 -----
1150 -- (a) Each transaction is its own coroutine.
1151 local function makeBankTransaction(txId, fraudScore)
1152     return coroutine.create(function()
1153         coroutine.yield(txId, "authenticated")
1154         coroutine.yield(txId, "transaction requested")
1155         -- (b) yield the fraud score at the appropriate stage
1156         coroutine.yield(txId, "fraud_score", fraudScore)
1157
1158         if fraudScore >= 70 then
1159             -- (c) pause for manual review instead of failing outright;
1160             -- resume() later feeds in the reviewer's decision
1161             local decision = coroutine.yield(txId, "PENDING manual review")
1162             while decision ~= "approve" and decision ~= "reject" do
1163                 decision = coroutine.yield(txId, "still_pending_review")
1164             end
1165             if decision == "reject" then
1166                 coroutine.yield(txId, "REJECTED by reviewer")
1167                 return -- (d) rejected transaction terminates here
1168             end
1169         end
1170
1171         coroutine.yield(txId, "authorized")
1172         coroutine.yield(txId, "confirmed") -- (d) successful transaction terminates here
1173     end)
1174 end
1175
1176 local function runQuestion17()
1177     divider("QUESTION 17: Fraud-Aware Mobile Banking Workflow")
1178     local txns = {
1179         {id = "BTX-1", co = makeBankTransaction("BTX-1", 20)}, -- low risk
1180         {id = "BTX-2", co = makeBankTransaction("BTX-2", 85)}, -- flagged, will approve
1181         {id = "BTX-3", co = makeBankTransaction("BTX-3", 92)}, -- flagged, will reject
1182     }
1183     local reviewerDecision = [{"BTX-2"} = "approve", [{"BTX-3"} = "reject"]
1184     local pending = {}
1185
1186     local finished = 0
1187     while finished < #txns do
1188         finished = 0
1189         for _, t in ipairs(txns) do
1190             if coroutine.status(t.co) == "dead" then
1191                 finished = finished + 1
1192             else
1193                 local ok, id, status, extra = coroutine.resume(t.co, pending[t.id])
1194                 pending[t.id] = nil
1195                 if status == "PENDING manual review" or status == "still_pending_review" then
1196                     -- simulate the human reviewer eventually making a call
1197                     pending[t.id] = reviewerDecision[id]
1198                     print(string.format("%s: %s (reviewer will say '%s')", id, status, reviewerDecision[id]))
1199                 else
1200                     print(string.format("%s: %s%s", id, status, extra and (" (" .. tostring(extra) .. ")") or ""))
1201                 end
1202                 if coroutine.status(t.co) == "dead" then finished = finished + 1 end
1203             end
1204         end
1205     end
1206 end
1207 --[[
1208 (e) A long-lived workflow like this can be waiting on a human reviewer
for seconds, minutes or hours - it would be wasteful and impractical to

```

```

1209     block a whole program (or spin a thread) just to wait. A coroutine can
1210     yield at "PENDING_manual_review" and consume zero CPU while suspended;
1211     the program can go on to serve entirely different requests, and only
1212     when the reviewer's decision actually arrives does the scheduler
1213     resume that exact transaction with the answer, picking up precisely
1214     where it paused.
1215     ]]
1216 end
1217
1218 -----
1219 -- QUESTION 18: Multi-Stage E-Commerce Fulfilment System (20 Marks)
1220 -----
1221 local STAGES_Q18 = {"validation", "payment", "warehouse_allocation", "packaging", "courier_assignment", "dispatch"}
1222
1223 -- (a) Each order is a coroutine with persistent state (its stage + outcome).
1224 local function makeOrder(orderId, failAtStage)
1225     return coroutine.create(function()
1226         for _, stage in ipairs(STAGES_Q18) do
1227             if stage == failAtStage then
1228                 -- (c) cancel immediately on an unrecoverable condition -
1229                 -- do not proceed to any later stage
1230                 coroutine.yield(orderId, "CANCELLED_at_" .. stage)
1231                 return
1232             end
1233             -- (b) progress one stage at a time, yielding after each
1234             coroutine.yield(orderId, stage)
1235         end
1236         coroutine.yield(orderId, "DISPATCHED")
1237     end)
1238 end
1239
1240 local function runQuestion18()
1241     divider("QUESTION 18: Multi-Stage E-Commerce Fulfilment")
1242     -- (d) central scheduler handling five+ orders interleaved
1243     local orders = {
1244         {id = "ORD-1", co = makeOrder("ORD-1", nil)},
1245         {id = "ORD-2", co = makeOrder("ORD-2", "payment")},
1246         {id = "ORD-3", co = makeOrder("ORD-3", nil)},
1247         {id = "ORD-4", co = makeOrder("ORD-4", "warehouse_allocation")},
1248         {id = "ORD-5", co = makeOrder("ORD-5", nil)},
1249     }
1250     local outcome = {}
1251     local finished = 0
1252     while finished < #orders do
1253         finished = 0
1254         for _, o in ipairs(orders) do
1255             if coroutine.status(o.co) == "dead" then
1256                 finished = finished + 1
1257             else
1258                 local ok, id, status = coroutine.resume(o.co)
1259                 if ok then
1260                     print(string.format("%s: %s", id, status))
1261                     outcome[id] = status
1262                 else
1263                     print(string.format("%s: FAILED (%s)", id, status))
1264                     outcome[id] = "FAILED"
1265                 end
1266                 if coroutine.status(o.co) == "dead" then finished = finished + 1 end
1267             end
1268         end
1269     end
1270
1271     -- (e) final summary of completed, failed and cancelled orders
1272     local completed, cancelled, failed = 0, 0, 0
1273     for id, status in pairs(outcome) do
1274         if status == "DISPATCHED" then completed = completed + 1
1275         elseif status:match("^CANCELLED") then cancelled = cancelled + 1
1276         else failed = failed + 1 end
1277     end
1278     print(string.format("\nSUMMARY: %d completed, %d cancelled, %d failed", completed, cancelled, failed))
1279 end
1280
1281 -----
1282 -- QUESTION 19: Fair Scheduling of Digital Government Services (20 Marks)
1283 -----
1284 -- (a) Each service queue is its own coroutine.
1285 local function makeServiceQueue(name, arrivalsPerRound)
1286     return coroutine.create(function()
1287         local queueSize = 0
1288         while true do
1289             queueSize = queueSize + arrivalsPerRound
1290             -- (b) yield queue size and progress info
1291             local served = coroutine.yield(name, queueSize)
1292             if served then queueSize = math.max(0, queueSize - served) end
1293         end
1294     end)
1295 end
1296
1297 local function runQuestion19()
1298     divider("QUESTION 19: Fair Scheduling of Digital Government Services")
1299     local services = {
1300         {name = "BirthCert", co = makeServiceQueue("BirthCert", 2), waitTurns = 0},
1301         {name = "Passport", co = makeServiceQueue("Passport", 15), waitTurns = 0}, -- much higher demand
1302         {name = "BizReg", co = makeServiceQueue("BizReg", 3), waitTurns = 0},

```

```

1303     {name = "LicenceRenewal", co = makeServiceQueue("LicenceRenewal", 4), waitTurns = 0},
1304 }
1305 local CAPACITY_PER_ROUND = 12 -- total requests the platform can serve per round
1306
1307 for round = 1, 8 do
1308     local sizes = {}
1309     for _, s in ipairs(services) do
1310         local ok, name, size = coroutine.resume(s.co, s.lastServed)
1311         s.lastServed = nil
1312         sizes[s.name] = size
1313     end
1314
1315     -- (c) score combines raw demand (queue size) AND how long a service
1316     -- has been waiting - not demand alone - so Passport's huge queue
1317     -- doesn't automatically claim all the capacity every single round.
1318     table.sort(services, function(a, b)
1319         local scoreA = sizes[a.name] + a.waitTurns * 10
1320         local scoreB = sizes[b.name] + b.waitTurns * 10
1321         return scoreA > scoreB
1322     end)
1323
1324     local remainingCapacity = CAPACITY_PER_ROUND
1325     print(string.format("-- Round %d --", round))
1326     for _, s in ipairs(services) do
1327         local queueSize = sizes[s.name]
1328         local share = math.min(queueSize, math.max(1, math.floor(remainingCapacity / 4)))
1329         share = math.min(share, remainingCapacity)
1330         if share > 0 then
1331             s.lastServed = share
1332             remainingCapacity = remainingCapacity - share
1333             s.waitTurns = 0
1334             print(string.format("  %-14s queue=%-4d served=%-3d", s.name, queueSize, share))
1335         else
1336             s.waitTurns = s.waitTurns + 1
1337             print(string.format("  %-14s queue=%-4d served=0 (waiting %d rounds)", s.name, queueSize,
1338 s.waitTurns))
1339         end
1340     end
1341 end
1342 --[[
1343 (d) The waitTurns * 10 term is deliberately large relative to normal
1344 queue-size differences, so any service that keeps getting a zero share
1345 quickly rockets up the priority order regardless of how small its raw
1346 queue is - this is what stops BirthCert or BizReg being permanently
1347 crowded out by Passport, while Passport's genuinely larger demand still
1348 lets it claim more of the capacity in an ordinary round.
1349
1350 (e) Weakness: the even four-way capacity split (remainingCapacity / 4)
1351 ignores how many services actually still have anything in their queue,
1352 so on a round where only two services have demand, capacity is still
1353 divided as if there were four - wasting throughput. An improvement
1354 would be to only divide remaining capacity among services that still
1355 have a non-zero queue this round, reallocating what an empty queue
1356 doesn't use back into the pool for the busier services.
1357 ]]
1358 end
1359
1360 -----
1361 -- QUESTION 20: Integrated National Emergency Response System (20 Marks)
1362 -----
1363 local STAGES_Q20 = {"report_received", "incident_verified", "resources_assigned", "response_underway",
1364 "incident_resolved"}
1365 local SEVERITY_WEIGHT = {high = 100, medium = 30, low = 5}
1366
1367 -- (a) Each incident is an independent coroutine yielding after every stage.
1368 local function makeIncident(incidentId, county, incidentType, severity, resourcesNeeded)
1369     return coroutine.create(function()
1370         local resourcesGranted = 0
1371         for _, stage in ipairs(STAGES_Q20) do
1372             if stage == "resources_assigned" then
1373                 -- (b) two-way communication: scheduler sends back how many
1374                 -- resource units it is allocating via resume(units)
1375                 local granted = coroutine.yield(incidentId, stage, "awaiting_allocation")
1376                 resourcesGranted = granted or 0
1377                 if resourcesGranted < resourcesNeeded then
1378                     -- not enough resources yet - stay "partially handled"
1379                     -- rather than being abandoned; keep asking each round
1380                     while resourcesGranted < resourcesNeeded do
1381                         local more = coroutine.yield(incidentId, "resources_assigned", "insufficient_" ..
1382 resourcesGranted .. "/" .. resourcesNeeded)
1383                         resourcesGranted = resourcesGranted + (more or 0)
1384                     end
1385                 end
1386                 coroutine.yield(incidentId, stage, "fully_resourced")
1387             else
1388                 coroutine.yield(incidentId, stage)
1389             end
1390         end
1391         coroutine.yield(incidentId, "RESOLVED")
1392     end)
1393 end
1394
1395 local function runQuestion20()
1396     divider("QUESTION 20: Integrated National Emergency Response System")

```

```

1397 local TOTAL_RESOURCE_POOL = 20
1398 local incidents = {
1399     {id = "INC-1", county = "Nairobi", severity = "high", needed = 12, waitTurns = 0},
1400     {id = "INC-2", county = "Kisumu", severity = "low", needed = 4, waitTurns = 0},
1401     {id = "INC-3", county = "Nakuru", severity = "medium", needed = 6, waitTurns = 0},
1402     {id = "INC-4", county = "Mombasa", severity = "high", needed = 10, waitTurns = 0},
1403 }
1404 for _, inc in ipairs(incidents) do
1405     inc.co = makeIncident(inc.id, inc.county, "generic", inc.severity, inc.needed)
1406     inc.resourceInput = nil
1407 end
1408
1409 local finished = 0
1410 local round = 0
1411 while finished < #incidents and round < 10 do
1412     round = round + 1
1413     -- (c) score by severity weight + waitTurns (ageing) - available
1414     -- resource pool constrains how much can actually be granted
1415     table.sort(incidents, function(a, b)
1416         return (SEVERITY_WEIGHT[a.severity] + a.waitTurns * 5) > (SEVERITY_WEIGHT[b.severity] + b.waitTurns * 5)
1417     end)
1418
1419     local pool = TOTAL_RESOURCE_POOL
1420     finished = 0
1421     for _, inc in ipairs(incidents) do
1422         -- (d) check status/return value before resuming, remove safely
1423         if coroutine.status(inc.co) == "dead" then
1424             finished = finished + 1
1425         else
1426             local grantAmount = nil
1427             if inc.resourceInput == "need" then
1428                 grantAmount = math.min(3, pool) -- allocate in modest chunks so no one incident hogs the pool
1429                 pool = pool - grantAmount
1430                 inc.waitTurns = grantAmount > 0 and 0 or (inc.waitTurns + 1)
1431             end
1432             local ok, id, stage, detail = coroutine.resume(inc.co, grantAmount)
1433             if not ok then
1434                 print(string.format("%s: ERROR -> %s", inc.id, id))
1435             else
1436                 print(string.format("%s (%s,%s): %s %s", id, inc.county, inc.severity, stage, detail or ""))
1437                 if stage == "resources_assigned" then
1438                     inc.resourceInput = "need"
1439                 else
1440                     inc.resourceInput = nil
1441                 end
1442             end
1443             if coroutine.status(inc.co) == "dead" then finished = finished + 1 end
1444         end
1445     end
1446 end
1447 --[[
1448 (e) Lower-severity incidents (INC-2, "low") don't get starved because
1449 waitTurns * 5 is added directly into their scheduling score every round
1450 they are skipped - after enough rounds a "low" incident's boosted score
1451 can exceed a "high" incident's base score, forcing the scheduler to
1452 finally serve it, even though high-severity incidents are still
1453 favoured by default.
1454
1455 (f) This program is quasi-concurrent/cooperative, not truly parallel:
1456 at any given instant exactly one incident's coroutine is executing,
1457 and it only ever hands control back at a coroutine.yield() call it
1458 chose to make - the scheduler cannot forcibly interrupt an incident
1459 mid-computation the way a real OS scheduler pre-empts a thread. This
1460 is perfectly adequate for representing WORKFLOW STATE (which stage an
1461 incident is in, how many resources it has), but it cannot model true
1462 simultaneous physical action - e.g. two rescue teams physically
1463 working on two different incidents at the exact same second. For that
1464 kind of genuine parallel, real-world execution (multiple response
1465 teams acting independently in real time), actual OS threads or
1466 separate processes - each with its own CPU core - would be the
1467 appropriate tool, not cooperative coroutines on a single Lua thread.
1468 ]]
1469 end
1470
1471 -----
1472 -- MAIN DRIVER
1473 -----
1474 local function main()
1475     runQuestion1()
1476     runQuestion2()
1477     runQuestion3()
1478     runQuestion4()
1479     runQuestion5()
1480     runQuestion6()
1481     runQuestion7()
1482     runQuestion8()
1483     runQuestion9()
1484     runQuestion10()
1485     runQuestion11()
1486     runQuestion12()
1487     runQuestion13()
1488     runQuestion14()
1489     runQuestion15()
1490     runQuestion16()

```

```
1491     runQuestion17()
1492     runQuestion18()
1493     runQuestion19()
1494     runQuestion20()
1495     print("\nAll 20 exercises executed.")
end

main()
```